Which LLM wrote this text? Fingerprinting LLM families with a CNN and an RNN

Anish Chitnis, Penn State. Individual project.

Code and results: https://github.com/anishhchitnis/llm-identifier

Summary

I trained a CNN and a BiLSTM to guess which LLM family wrote an answer. The data has five families, which are GPT, Llama 2, Bard, Falcon and MPT. When the models see only the answer text, they get about 0.74 (CNN) and 0.71 (BiLSTM) macro F1 on a test set where random guessing gives 0.20. A simple TF IDF and logistic regression baseline did slightly better, at 0.77. When the models see only the prompt, they are close to chance at 0.22. Adding the prompt to the answer did not help. I also found that the small gap above chance for prompt only models comes from a data quirk and not from real fingerprints in the prompt.

1. Problem definition and dataset

1.1 The problem

Many LLMs write fluent text, so it is hard to tell by eye who wrote what. The task here is to look at a text and predict the LLM family that produced it. This is a 5 class classification problem. I answer these two research questions.

- **RQ1.** Can we identify the LLM family from the output text alone?
- **RQ2.** How does the result change if the model sees only the input (the prompt), only the output, or both?

1.2 Where the data comes from

I used the public UltraFeedback dataset from Hugging Face (openbmb/UltraFeedback, MIT license). It has 63,967 prompts. Each prompt has four answers, written by four of 17 different models. Each row has a prompt, an answer, the name of the model that wrote it, and the name of the source dataset the prompt came from. This fits the project format directly, because each row is (LLM name, input, output). I did not need any API keys, so all data collection was free.

I grouped the models into families. GPT is GPT 4 and GPT 3.5 turbo. Llama 2 is the 7B, 13B and 70B chat models. Bard, Falcon (40B instruct) and MPT (30B chat) each have one model. That gives five families, which is more than the three the project asks for. I used the source dataset of each prompt as its domain. There are four domains, which are Evol Instruct, Flan, ShareGPT and UltraChat. The Flan source is split into several parts in the data, and I merged them into one domain.

1.3 Cleaning and balancing

- I kept answers between 20 and 8000 characters and dropped duplicates.
- I capped every family and domain pair at 1000 rows, so no single family or domain can dominate. Bard only has 701 Flan answers in the whole dataset, so that one pair has 701 rows.
- Some answers name the model, for example "As ChatGPT" or "I am OpenAI". A classifier could just read the name. I found these with a pattern match and removed them. This was about 1 percent of the rows, or 183 rows in total.
- I split the data into train, validation and test with an 80, 10, 10 split. The split is grouped by prompt text, so the same prompt never appears in two splits. I checked that the overlap is zero. This matters because answers to the same prompt are alike, and a random split would leak information.

Family	Rows kept	Mean words	Median words	Self naming rows removed
Bard	3,701	268	245	38
Falcon	4,000	103	70	31
GPT	4,000	226	216	52
Llama 2	4,000	267	239	34
MPT	4,000	194	161	28
Total	19,701			183

Table 1. The dataset before the leak rows were removed. Falcon answers are much shorter than the others.

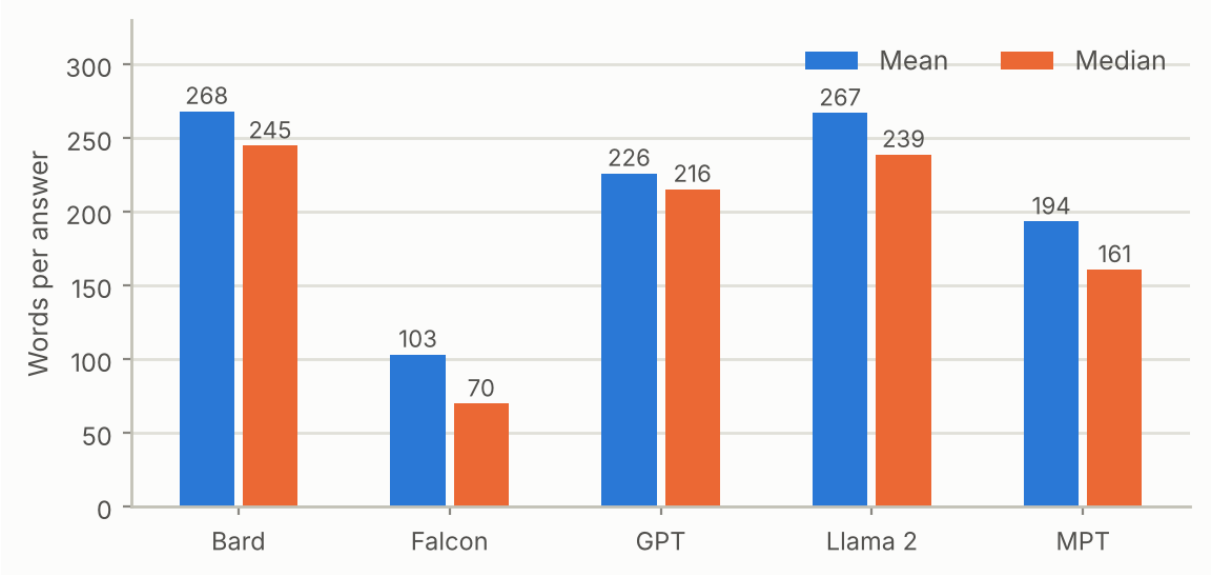


Figure 1. Mean and median answer length for each family. Length is one clear difference between families.

Group	Train	Validation	Test
Bard	2,928	357	378
Falcon	3,206	376	387
GPT	3,071	443	434
Llama 2	3,219	395	352
MPT	3,207	385	380
Evol Instruct	3,937	524	487
Flan	3,767	471	440
ShareGPT	3,920	494	516
UltraChat	4,007	467	488
All rows	15,631	1,956	1,931

Table 2. Rows in each split. All rows is 19,518 after the leak rows were removed.

2. Models and training

2.1 Input to the networks

Both networks use the same tokenizer. It splits text into words, punctuation marks and newlines. It does not lowercase, because capital letters and formatting like bullet points and line breaks can be part of a model's style. The vocabulary has the 30,000 most common tokens from the training set only. Each answer is cut or padded to 384 tokens. For RQ2, the input and output are joined into one sequence. The input is cut to 128 tokens, then a separator token, then the output.

2.2 The CNN

The CNN is a TextCNN. It has a 200 dimension embedding layer, then three parallel 1D convolutions with kernel sizes 3, 4 and 5 and 128 filters each. After each convolution it uses a ReLU and a max pool over the whole text. Padding is masked so it cannot win the max pool. The three pooled vectors are joined, passed through dropout of 0.5, and then a linear layer gives five scores. It has about 6.31 million parameters.

2.3 The RNN

The RNN is a bidirectional LSTM with hidden size 128 in each direction and the same 200 dimension embedding. It uses packed sequences so padding is ignored. I join the mean pool and the max pool of the hidden states, apply dropout of 0.5, and use a linear layer. It has about 6.34 million parameters.

2.4 Training details

Setting	Value
Optimizer	Adam, learning rate 0.001
Batch size	64
Loss	Cross entropy with class weights
Gradient clipping	1.0
Max epochs	20
Early stopping	Stop when validation macro F1 does not improve for 3 epochs, keep the best epoch
Seeds	3 seeds for every neural result. I report mean and standard deviation
Hardware	One T4 GPU on Google Colab

Table 3. Training settings, the same for the CNN and the BiLSTM.

I only used validation data to pick the best epoch and to pick the regularization strength C for the baselines. The test set was used once per final model. Macro F1 is my main number because it treats all five families the same. I also report accuracy.

2.5 Baselines

I added three reference points. The first is a majority class guess. The second is TF IDF with word 1 and 2 grams plus logistic regression. The third is TF IDF with character 2 to 5 grams plus logistic regression. The character version keeps case, punctuation and newlines. These baselines help show how much the neural networks add.

3. Results

3.1 RQ1, output only

Model	Test accuracy	Test macro F1
Majority class	0.182	0.062
TF IDF words + logistic regression	0.772	0.773
TF IDF characters + logistic regression	0.767	0.769
CNN (3 seeds)	0.737 ± 0.006	0.743 ± 0.005
BiLSTM (3 seeds)	0.714 ± 0.037	0.708 ± 0.042

Table 4. RQ1 results on the test set with 384 tokens of the output. Random guessing among five classes is 0.20 accuracy.

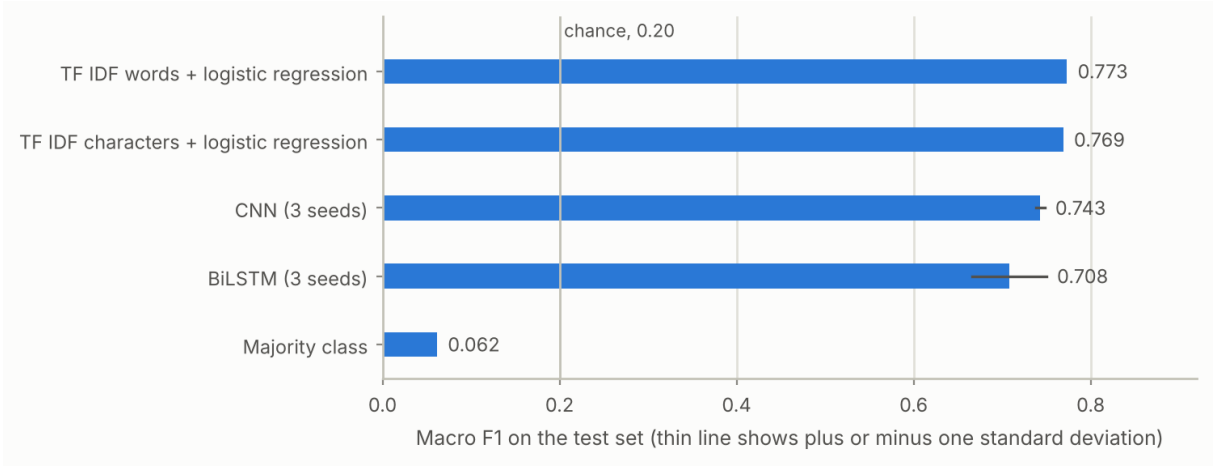


Figure 2. Test macro F1 for each model. The grey line is chance at 0.20.

The answer to RQ1 is yes. Every real model is far above the 0.20 chance line. The CNN reaches 0.743 macro F1 and the BiLSTM reaches 0.708. The TF IDF baselines are a bit better at 0.773 and 0.769. So on this data, the neural networks do not beat a simple linear model on word counts. I think this is because the style signals are strong and local, like the words "Certainly" or "happy to help", and a linear model can use those well. The CNN is 3.5 points ahead of the BiLSTM, but the BiLSTM changes a lot between seeds, as Table 5 shows, so I would not call that gap firm.

Model	Seed	Test accuracy	Test macro F1	Epochs run
CNN	0	0.744	0.749	8
CNN	1	0.733	0.739	12
CNN	2	0.735	0.740	11
BiLSTM	0	0.679	0.668	11
BiLSTM	1	0.752	0.752	20
BiLSTM	2	0.712	0.703	12

Table 5. Each seed for the output only models at 384 tokens. BiLSTM seed 1 ran all 20 epochs. The other runs stopped early.

3.2 Which families are easy and which are hard

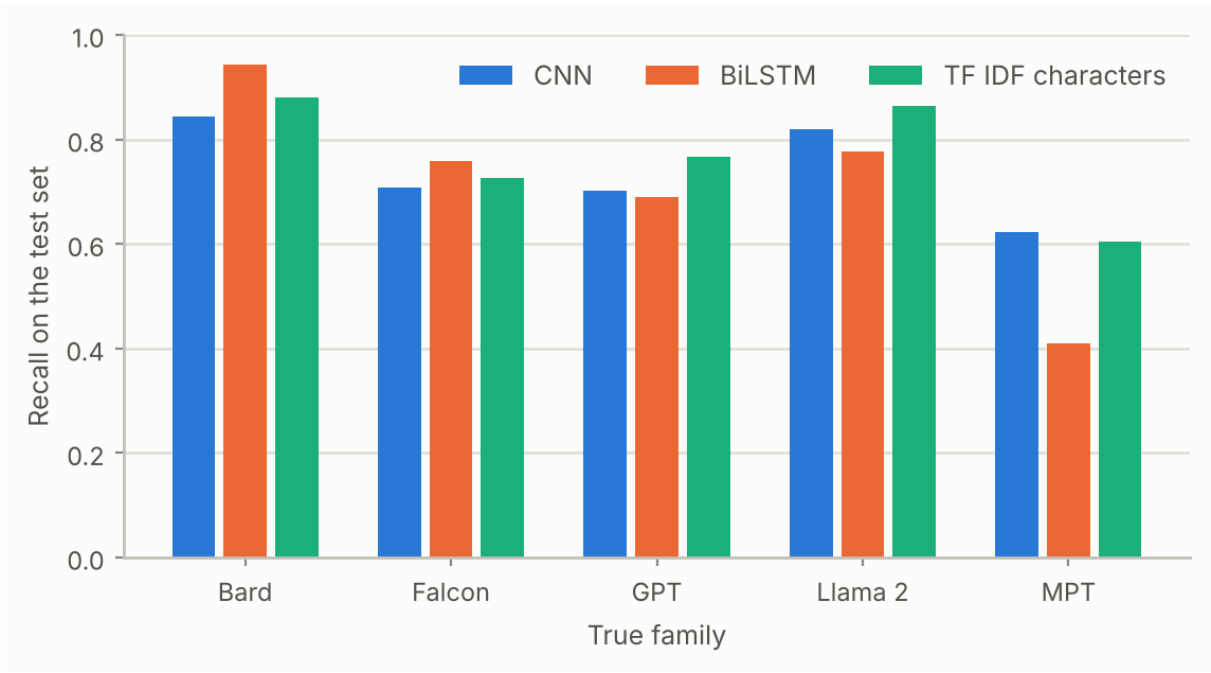


Figure 3. Recall for each true family. Recall is the share of that family's answers that were found.

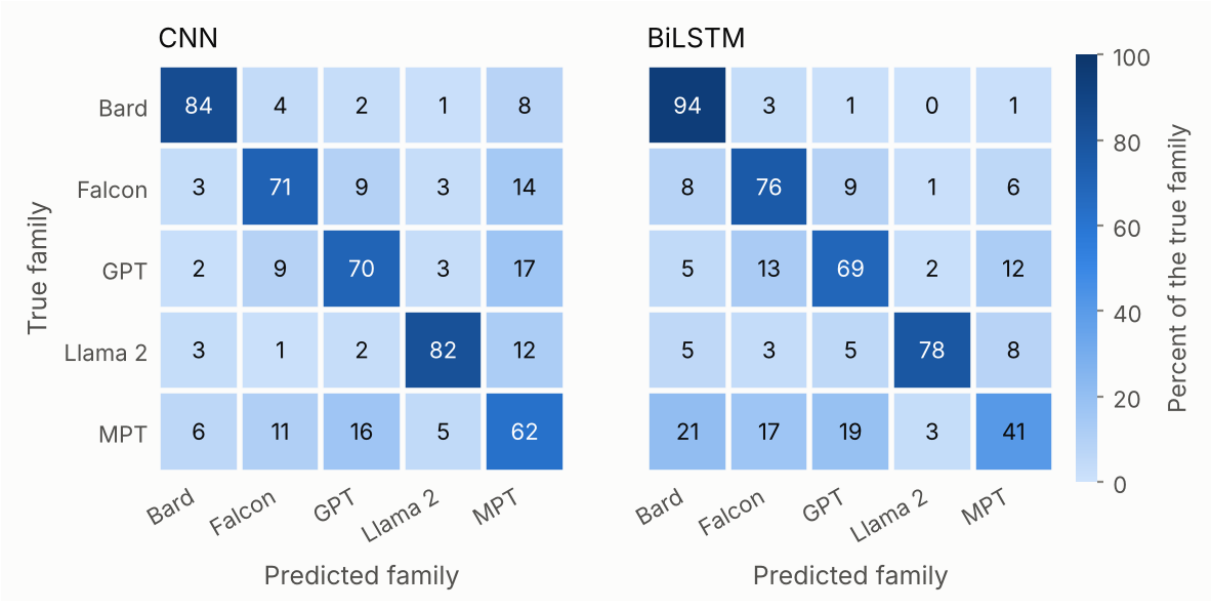


Figure 4. Confusion matrices for the CNN and the BiLSTM. Each row adds to 100 percent. The diagonal is the correct answers.

Bard is the easiest family. The BiLSTM finds 94 percent of Bard answers and the CNN finds 85 percent. Llama 2 is next. MPT is the hardest. The CNN finds 62 percent of MPT answers and the BiLSTM only 41 percent. MPT answers are most often mistaken for GPT and Falcon. GPT and Falcon are also mixed up with each other. The BiLSTM sent many MPT answers to Bard, which is why its MPT recall is so low.

4. Analysis and experiments

4.1 How much text does the model need? (truncation)

I trained both networks again with the output cut to 16, 32, 64, 128, 256 and 384 tokens. Each setting used 3 seeds.

Tokens kept	CNN macro F1	BiLSTM macro F1
16	0.567 ± 0.011	0.561 ± 0.009
32	0.599 ± 0.011	0.587 ± 0.017
64	0.657 ± 0.015	0.633 ± 0.022
128	0.717 ± 0.010	0.707 ± 0.025
256	0.737 ± 0.002	0.727 ± 0.008
384	0.743 ± 0.005	0.708 ± 0.042

Table 6. Test macro F1 when only the first N tokens of the answer are kept.

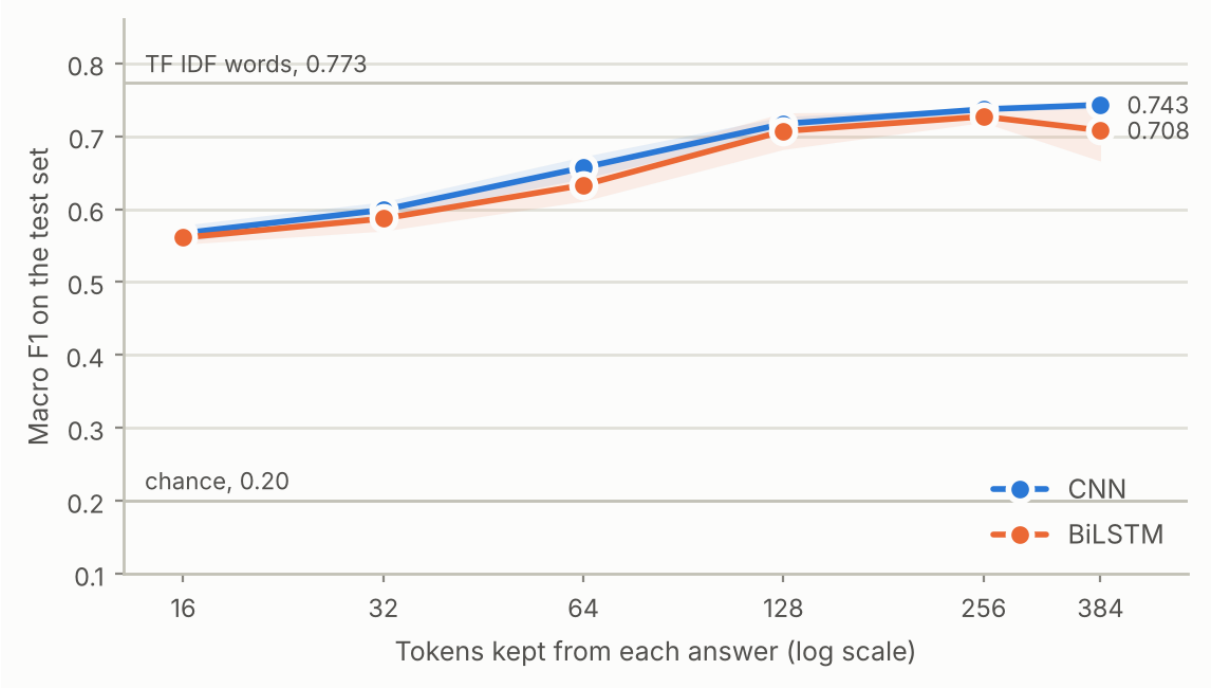


Figure 5. Macro F1 against tokens kept. The shaded band is one standard deviation. The grey line at the top is the best TF IDF baseline.

Even 16 tokens, which is about one short sentence, gives 0.567 for the CNN and 0.561 for the BiLSTM. That is almost three times chance. This means the start of an answer already shows a lot of style, for example how a model begins with "Sure" or "Certainly". The score rises quickly until about 128 tokens and then flattens. The CNN stays slightly ahead of the BiLSTM at every length, but at 16 tokens they are about the same. The BiLSTM at 384 is lower than at 256, but its spread is large, so I would not read much into that drop.

4.2 RQ2, input only, output only, or both

Model	Output only	Input only	Input + output
CNN	0.743 ± 0.005	0.219 ± 0.004	0.732 ± 0.010
BiLSTM	0.708 ± 0.042	0.217 ± 0.008	0.704 ± 0.007
TF IDF words	0.773	0.226	0.722

Table 7. Test macro F1 for RQ2. Chance is 0.20.

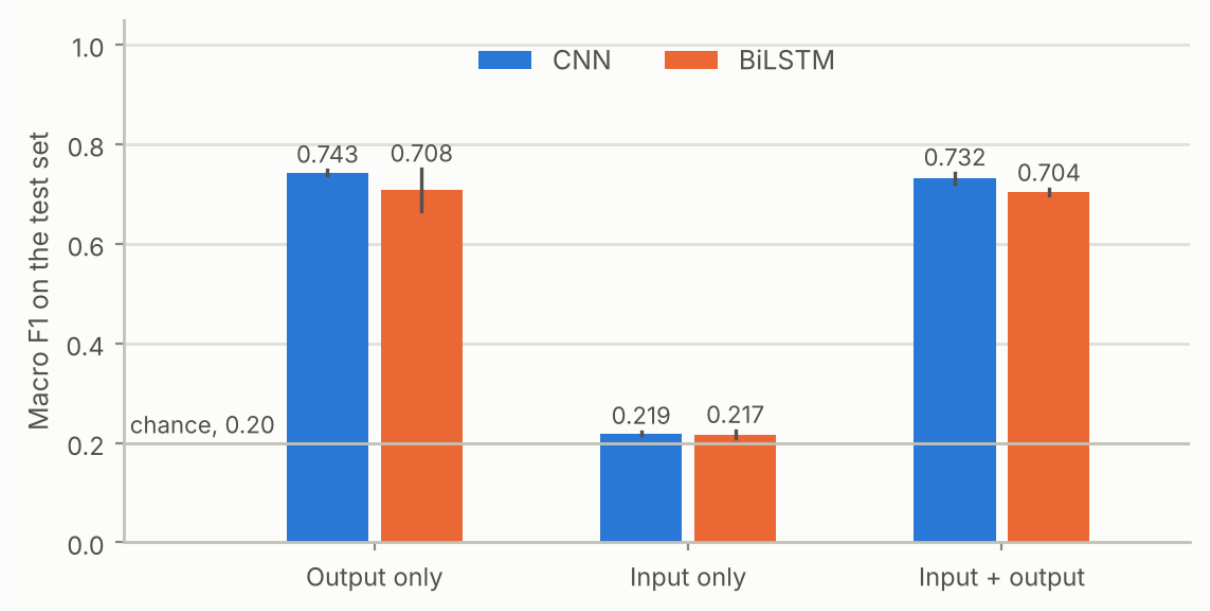


Figure 6. RQ2 results. The grey line is chance at 0.20.

- **Output only** works well, as in RQ1.
- **Input only** is near chance, at 0.219 for the CNN and 0.217 for the BiLSTM. This is what I expected, because the prompt is written before the model answers.
- **Input and output together** is not better than output only. The CNN goes from 0.743 to 0.732 and the BiLSTM from 0.708 to 0.704. The TF IDF model drops more, from 0.773 to 0.722. The prompt adds noise and nothing useful.

So the answer to RQ2 is that the output carries almost all of the signal, and the input adds nothing.

4.3 Why is input only a little above chance?

Input only scores 0.22, a little above 0.20. The BiLSTM input only model also finds 55 percent of Bard answers, but only 12 to 22 percent of the answers from each other family. That looked strange, since the prompt should not say who answers. I looked for a cause.

The cause is a data quirk. Bard has only 701 Flan rows, while every other family has 1000 in each domain. This means that a Flan prompt is a bit more likely to be from a non Bard family overall. But Bard's Flan rows are not spread evenly over prompt lengths. I split the Flan prompts into four equal groups by length and measured the share of Bard rows in each. Bard's overall share of Flan rows is 15 percent, but it is 26 percent for the shortest prompts and only 4 percent for the second shortest.

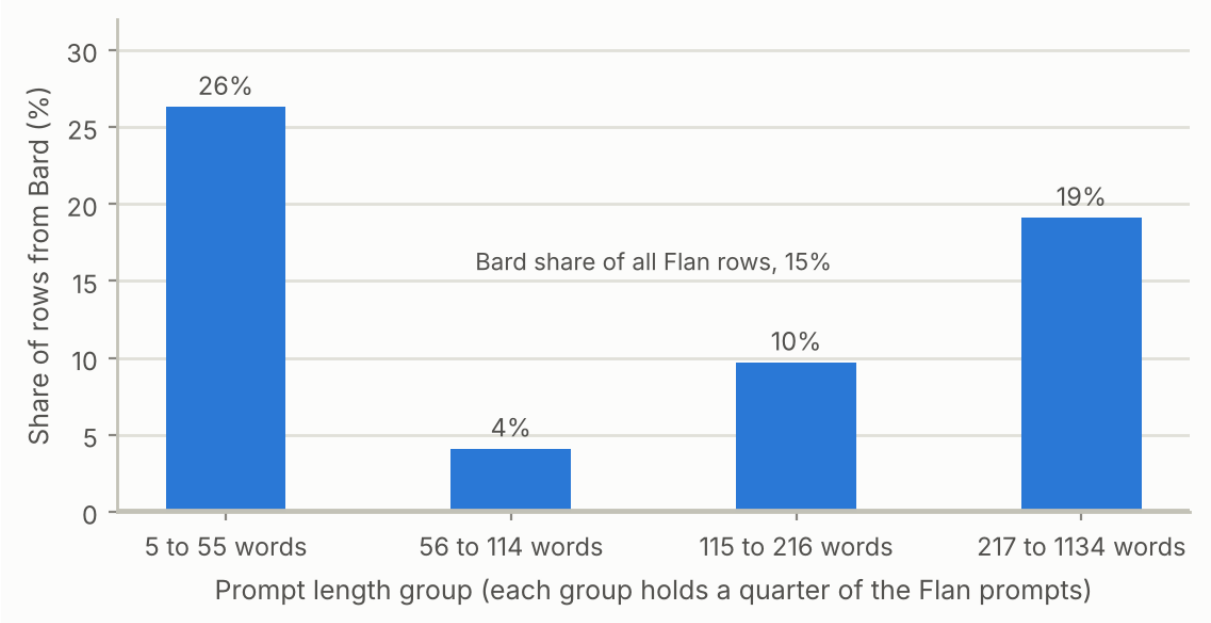


Figure 7. Bard share of Flan rows by prompt length group. If prompt length told us nothing, every bar would be near the 15 percent line.

Domain	Chi square	Degrees of freedom	p value
Evol Instruct	16.9	12	0.153
Flan	286.4	12	below 0.0001
ShareGPT	8.8	12	0.723
UltraChat	6.6	12	0.885

Table 8. Chi square test of prompt length group against family, for each domain. Only Flan shows a link.

Classifier that sees only prompt length and domain	Test accuracy	Test macro F1
Majority class	0.182	0.062
Domain only	0.182	0.133
Prompt length and domain	0.221	0.216

Table 9. A simple classifier that cannot read any words gets the same score as the neural input only models.

The chi square test is strongly significant for Flan and not for the other three domains. A simple classifier that only knows the prompt length and the domain gets 0.216 macro F1. That is the same as the CNN and BiLSTM input only results of 0.219 and 0.217. So the neural models are not finding any fingerprint in the prompt words. They found the Bard and Flan length pattern. This is a good example of a shortcut, and it shows why checking the data matters as much as the model.

4.4 What the baselines learned

The top logistic regression features match what I see in real chat output. Llama 2 uses exclamation marks and phrases like "happy to help" and "respectful". Bard uses bullet stars and blank lines. MPT often says "Certainly" and ends with "Overall". GPT uses phrases like "Confidence: 95", which probably come from the way some prompts ask for a confidence level. Falcon features include the words "assistant" and "user" and leftover tag pieces like "< /". Falcon answers are also much shorter, with a median of 70 words against 161 to 245 for the others.

Some of these features say more about the training data and chat template of each model than about the model's deep style. That is still a valid fingerprint, but it may be easy to hide. It also means these results may not carry over to other prompts.

5. Limits of this work

- All text comes from one dataset, UltraFeedback. The results may not hold for other prompts or for newer models.
- A family can have more than one model, such as GPT 4 and GPT 3.5, and Llama 2 in three sizes. I did not test whether the classifiers could tell models in the same family apart.
- In UltraFeedback each prompt gets four of the 17 models, so not every family answered every prompt.
- Bard has fewer rows than the other families because of how much Bard data exists in the source.
- I used 3 seeds, so the standard deviations are rough. I did not do a heavy search for hyperparameters.

6. Lessons and experience

The biggest thing I learned is that the data work took more thinking than the models. Early on I thought the hard part would be the CNN and the LSTM, but the CNN and the LSTM were short to write and trained without trouble. Getting the data right took much longer. I had to find that the Flan prompts were under a different name, so my first dataset only had three domains. I also had to find the self naming answers where a model says its own name, since a classifier would just read that and look smart for the wrong reason.

I also learned to split by prompt. At first it seemed fine to split rows at random, but then the same prompt would be in train and test with different answers, and the scores would be too high. Grouping by prompt fixed that.

A surprise was that the simple TF IDF baseline beat my neural networks. At first I felt bad about that, but I think it is a useful result. It told me that the style signal is mostly in which words and symbols appear, and not in long range structure. If I had skipped the baseline I would have thought 0.74 was great without knowing that a much simpler model gets 0.77.

The input only result taught me the most. A score of 0.22 looked tiny, and I almost wrote it off as noise. But it was consistent across both models, so I dug in. The real cause was that Bard has fewer Flan rows. It would have been easy to claim that the prompt leaks a bit of information about the model. That would have been wrong. Next time I will check simple explanations before I make a claim.

On the practical side, next time I will save and download result files after every step. If I had more time, I would test the models on a family that was not in training, and look at the wrong predictions one by one to see what the MPT answers that look like GPT have in common.

Reproducing this

All code is in the repo under src. The file run_all.sh runs the full pipeline, which is dataset building, preprocessing, baselines, the CNN and BiLSTM runs, the truncation runs, the RQ2 runs and the diagnostics. It needs a GPU and takes about an hour and a half on a Colab T4. The file src/make_figures.py draws the figures from the files in the results folder.